

Experiment 4 - Function Generator

Ali Dehghani,
810103414

Abstract— This report details the Verilog HDL design, synthesis, and simulation of an Arbitrary Function Generator (AFG) system. The architecture integrates a Direct Digital Synthesis (DDS) unit utilizing a single-quadrant sine ROM, a counter-driven waveform generator for periodic shapes, a 9-bit programmable clock divider for frequency tuning, and a digital amplitude selector. Both pre-synthesis functional analysis and post-synthesis gate-level netlist simulations were conducted in ModelSim to verify system reliability using Intel Quartus II targeting an FPGA board.

Keywords— Verilog HDL, FPGA, Direct Digital Synthesis (DDS), Function Generator, Clock Divider, Post-Synthesis Simulation.

I. INTRODUCTION

This experiment focuses on the design, hierarchical integration, and hardware-level simulation of an Arbitrary Function Generator (AFG) using Verilog HDL. The primary objective is to construct a unified digital architecture capable of producing multiple periodic waveforms—including sine, square, triangle, reciprocal, and rectified signals—with dynamically adjustable frequencies and scaling amplitudes. This study bridges the gap between ideal logical behavior and real-world hardware by utilizing Intel Quartus II for block-schematic synthesis targeting an FPGA device. By evaluating the complete system through both behavioral and post-synthesis netlist simulations in ModelSim, the experiment provides a comprehensive methodology for validating multi-module signal generation under realistic hardware constraints.

II. WAVEFORM GENERATOR

A. DDS Module Implementation:

The Direct Digital Synthesis (DDS) module is the core component for generating high-precision sine waves. The implementation relies on three key sub-modules:

- 1) *Phase Accumulator (datapath_ph)*: Acts as a counter to track the instantaneous phase of the sine wave. It iterates through 256 states (8-bit), ensuring a periodic signal.
- 2) *Phase Controller (controller_ph)*: This unit manages the four quadrants of the sine cycle. It uses the most significant bit of the phase to determine the signal's sign (positive or negative) and maps the remaining bits to the memory address of the look-up table (LUT).

- 3) *Sine ROM*: Stores the amplitude values of the first quadrant of a sine wave in a memory file (sine.mem). By reading these values and applying the symmetry properties (using the quadrant logic), the full sine wave is reconstructed. The output is then formatted into a 9-bit signal, ensuring no overflow occurs during the reconstruction of negative half-cycles.

B. Waveform Generator Implementation

The Waveform_Generator module serves as the multifunctional stage of the design. It takes the output of the DDS module and the system clock as references to produce various waveforms:

- 1) *Square Wave*: Generated by comparing the counter to a threshold (128) to ensure a 50% duty cycle.
- 2) *Triangle Wave*: Created using a linear ramp-up (counter value) and ramp-down (511 - counter value) logic.
- 3) *Reciprocal*: Computed through a division operation on the counter value.
- 4) *Rectified Waves*: The Full-wave and Half-wave rectified signals are derived by processing the DDS output. Full-wave rectification flips the negative amplitude values (by subtracting them from the baseline), while half-wave rectification clamps the negative portion to the zero-level (offset 128).

C. Simulation and Verification (Testbench)

To verify the design, a comprehensive testbench was developed.

Time Synchronization: The simulation timing was carefully calibrated based on the 50 MHz clock frequency. Each waveform cycle requires 256 clock pulses (5120 ns). By setting the testbench delay intervals to multiples of 25600 ns (5 full cycles), we ensured that each waveform is observed from its starting point to its completion, avoiding any partial or cut-off signals in the simulation output.

Verification: The testbench drives the rst, clk_en, and Sel signals to systematically cycle through all six waveform modes. The verification confirms that the outputs align perfectly with the theoretical expectation of a function generator, demonstrating correct integration between the DDS core and the waveform processing unit.

D. Simulation:

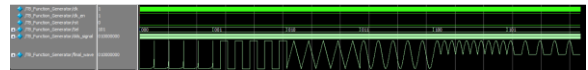


Fig. 1 Waveform generator waveform

III. FREQUENCY SELECTOR

A. Frequency Selector Implementation

To generate different output frequencies, I implemented a frequency selector using a 9-bit counter. The counter acts as a programmable frequency divider. The upper five bits of the counter are loaded from the input port PI[4:0], while the lower four bits are fixed to the binary value 0110.

Whenever the init signal is asserted, the counter is loaded with the value {PI, 4'b1000}. After initialization, the counter increments on every rising edge of the clock. When the counter reaches its maximum value (511), the output signal clkEn becomes active, indicating the completion of one division period. The counter is then reloaded with the programmed value and the process repeats continuously.

This structure allows the output frequency to be adjusted simply by changing the value of PI[4:0].

B. Testbench Description

To verify the operation of the frequency selector, I developed a testbench that applies different values to the PI[4:0] input. After the reset sequence, the init signal is asserted to load the desired division value into the counter.

The testbench was configured to test at least three different values of PI, corresponding to three different output frequencies. For each case, sufficient simulation time was provided to observe the generated clkEn pulses and measure the division period.

The signals monitored during simulation were clk, rst, init, PI[4:0], and clkEn. By comparing the measured periods for different values of PI, I verified that the output frequency changes according to the programmed divider value.

C. Simulation

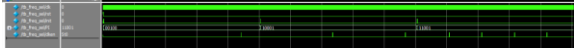


Fig. 2 Frequency selector waveform

As we can see from Fig.2 when I increase the input number value count of cycles that needed to clkEn become one decrease(because that need less clk cycles to reach 511). And if we want to verify the frequency selector operation, we have:

D. Verification of Correct Operation

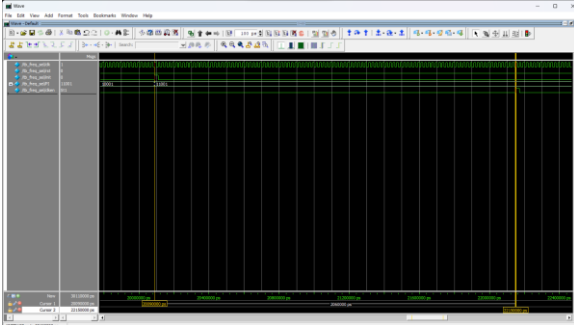


Fig. 3 Frequency selector waveform detail

In the last part of the testbench, I loaded the decimal value 25 into the 5-bit input PI[4:0]. Since the four least significant bits of the counter are fixed to the binary value 1000 (decimal 8), the loaded value becomes:

{11001,1000}

This operation is equivalent to shifting the value 25 four bits to the left and then adding 8. Therefore, the loaded counter value is:

$$25 \times 16 + 8 = 408$$

Since the counter generates clkEn when it reaches 511, the required number of clock cycles is:

$$511 - 408 = 103$$

According to Fig. 3, the time interval between loading the counter and the assertion of clkEn is approximately 2060 ns. Given that the clock period is 20 ns, the number of elapsed clock cycles is:

$$2060 \div 20 = 103$$

This exactly matches the theoretical calculation. Therefore, the simulation results confirm that the frequency selector operates correctly.

IV. AMPLITUDE SELECTOR

A. Amplitude Implementation

I write a module with two input one of them output of waveform generator and the another one AmpSel and based on the AmpSel I shift the input to the right and give it as output.

B. Testbench

I use all amplitude functions and also change the frequency selector input to see different range and amplitude for waveforms.

C. Simulation

By holding the Sel signal constant (e.g., selecting the Triangle waveform), the output amplitude was observed to decrease by powers of two as the AmpSel control signal changed.

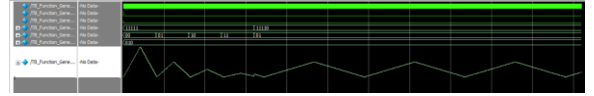


Fig. 4 Function generator waveform

V. THE TOTAL DESIGN

A. Quartus Synthesis and Top Module Creation

I created a new Top module in Quartus that integrated all sub-modules except the dds module. The DDS module relies on a ROM initialized via a .mem file (\$readmemb), which can complicate synthesis and direct netlist generation in some configurations. Therefore, I kept the dds module as a behavioral model.

After compiling the Top module in Quartus, I performed a full synthesis and timing analysis. I then exported the post-synthesis gate-level netlist (Verilog output) along with the Standard Delay Format (.sdo) file, which contains the precise routing and gate propagation delays calculated for the target FPGA.

Flow Summary	
Flow Status	Successful - Sun Jun 21 22:13:22 2026
Quartus Prime Version	20.1.0 Build 711 06/05/2020 S.J. Lite Edition
Revision Name	4
Top-level Entity Name	Top
Family	Cyclone IV E
Device	EP4CE6E22A7
Timing Models	Final
Total logic elements	201 / 6,272 (3 %)
Total registers	51
Total pins	32 / 92 (35 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)

Fig. 5 Frow summary

B. Testbench

To simulate the synthesized design, I developed a comprehensive testbench. Unlike the previous RTL testbench, this setup required instantiating two distinct modules:

- The post-synthesis netlist of the Top module.
- The behavioral dds module.

I applied the exact same input stimuli (reset, phase increments, and amplitude selections) as used in the ideal RTL simulation to ensure a direct comparison.

C. Simulation Output

Upon running the simulation with the delay files included, the resulting waveforms displayed noticeable differences compared to the ideal RTL simulation. Most prominently, the analog representation of the generated signals, specifically the triangular wave, exhibited multiple high-frequency voltage spikes or vertical lines (glitches).

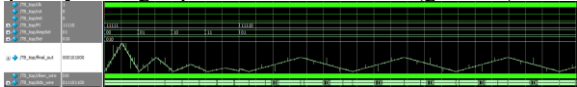


Fig. 6 Post synthesize waveform

D. Glitch Analysis

These transient spikes are a direct confirmation that the simulation is utilizing real hardware delays. In a synthesized multi-bit datapath, bits do not flip at the exact same picosecond due to unequal routing paths and varying logic gate delays. For instance, when a bus transitions between values that require multiple bit-flips (from 01111111 to 10000000), slight timing mismatches can cause the bus to momentarily hold an unintended value (like 11111111) for a fraction of a nanosecond. When plotted as an analog waveform, these brief intermediate states manifest as sharp, vertical glitches. This observation perfectly illustrates the physical realities of digital logic design and the necessity of proper setup/hold timing analysis.

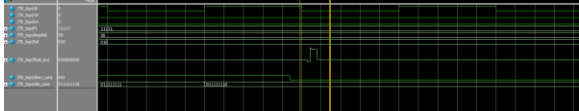


Fig. 7 Waveform glitch details